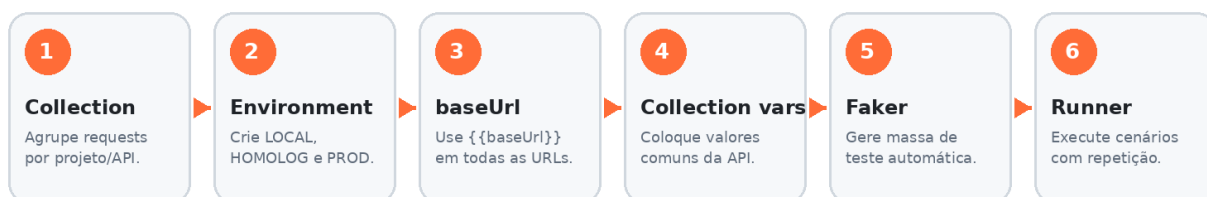


Guia de Estudo Postman

Variáveis, Ambientes, Base URL, Collection Variables e Dynamic Variables (Faker)

Fluxo de estudo recomendado



Mapa visual do fluxo recomendado para montar requests reutilizáveis no Postman.

Objetivo do material

Este PDF foi refeito do zero com base na documentação oficial do Postman. A ideia é servir como um roteiro de estudo prático: primeiro você entende o conceito, depois configura no Postman, depois testa com uma API pública segura para treino.

Vamos usar a Postman Echo API como API de teste, porque ela devolve na resposta os dados que você enviou. Isso facilita enxergar URL, query params, body, headers e variáveis funcionando.

1. Visão geral: por que usar variáveis?

Variável no Postman é um valor reutilizável. Em vez de repetir a mesma URL, token, versão da API ou usuário de teste em vários endpoints, você cria um nome e chama esse nome entre duas chaves: `{{nomeDaVariavel}}`. Na hora de enviar a request, o Postman troca o nome pelo valor real.

Analogia prática: pense na collection como uma pasta de endpoints, o ambiente como o endereço onde a API está rodando, e as variáveis como etiquetas que o Postman substitui pelo valor certo no momento do envio.

Exemplo simples:

URL sem variável:
`https://postman-echo.com/get`

URL com variável:
`{{baseUrl}}/get`

Se o ambiente LOCAL tiver `baseUrl = https://postman-echo.com`, a URL enviada será exatamente `https://postman-echo.com/get`.

Quando isso começa a fazer diferença?

- Quando você tem a mesma API rodando em LOCAL, HOMOLOG e PROD.
- Quando uma collection tem muitos endpoints usando a mesma base da URL.
- Quando você quer testar vários usuários, tokens, IDs ou massas de dados.
- Quando quer evitar erro de copiar e colar URL em dezenas de requests.

Fonte principal: documentação oficial do Postman sobre variáveis, ambientes e dynamic variables.

2. Conceitos essenciais

Termo	Para que serve	Exemplo correto
Collection	Agrupar requests relacionadas a uma API ou projeto.	ERP Hospitalar API
Environment	Guardar valores que mudam conforme o contexto.	LOCAL, HOMOLOG, PROD
Base URL	Guardar a parte fixa do endereço da API.	{{baseUrl}} = https://postman-echo.com
Collection variable	Guardar valores comuns a toda a collection.	versaoApi = v1
Dynamic variable / Faker	Gerar dados falsos e aleatórios em cada envio.	{{randomEmail}}

Atenção: Base URL não é endpoint. Base URL é só o começo do endereço. Endpoint é o caminho específico depois dela, como /get, /post, /pacientes ou /atendimentos.

Hierarquia de variáveis no Postman

Quando existem variáveis com o mesmo nome, a mais específica vence.

1. Global

Alcance mais amplo. Use pouco. Ex.: protótipos.

2. Collection

Vale para todos os endpoints da collection. Ex.: versaoApi = v1.

3. Environment

Muda conforme LOCAL, HOMOLOG ou PROD. Ex.: baseUrl.

4. Data

Vem de arquivo CSV/JSON no Runner. Ex.: massa de testes.

5. Local

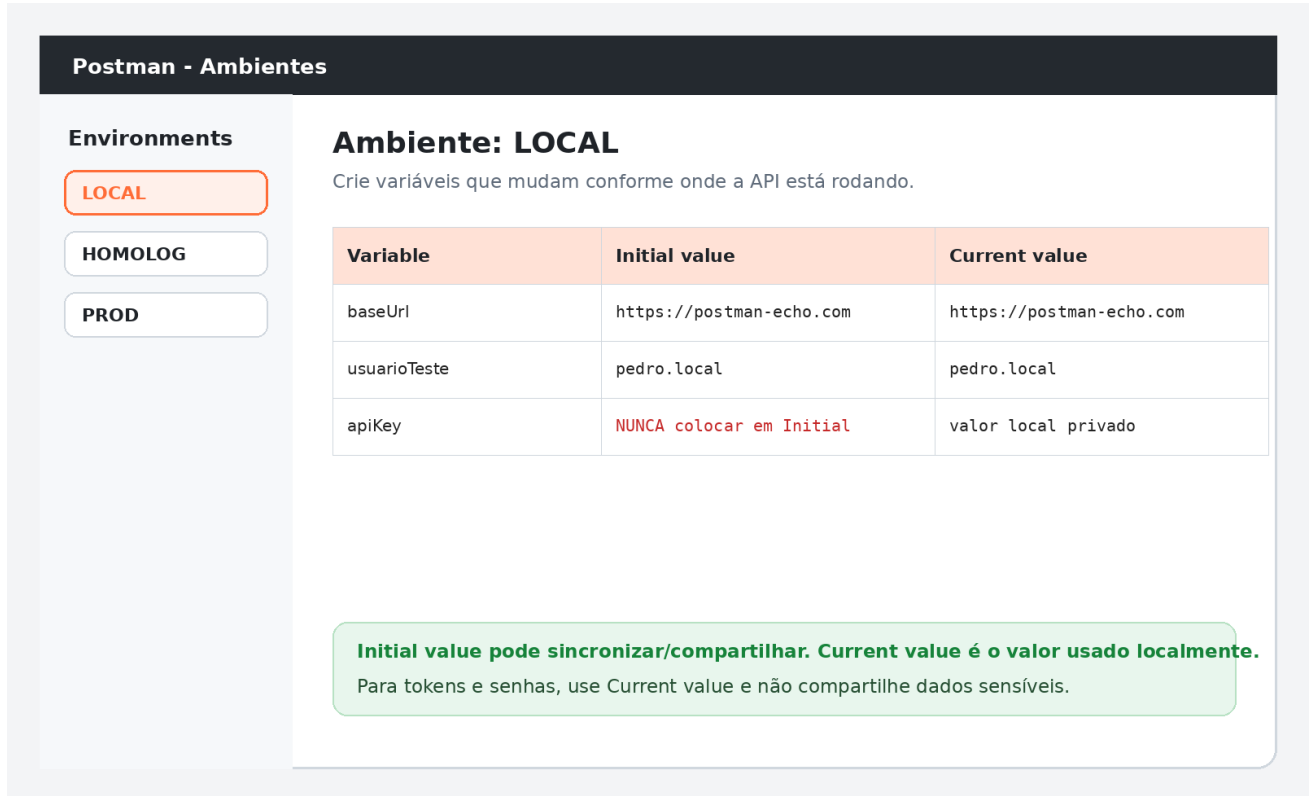
Criada durante execução/script. Mais específica. Ex.: tokenTemporario.

Dica prática: evite Global para projetos reais. Prefira Collection + Environment.

A hierarquia ajuda a entender qual valor será usado quando existe mais de uma variável com o mesmo nome.

3. Criando ambientes: LOCAL, HOMOLOG e PROD

Environment é um conjunto de variáveis que representa um contexto de execução. Em um projeto real, a mesma API costuma ter endereços diferentes: um para desenvolvimento local, outro para homologação e outro para produção.



The screenshot shows the Postman 'Ambientes' (Environments) interface. On the left, under 'Environments', there are three buttons: 'LOCAL' (highlighted in orange), 'HOMOLOG', and 'PROD'. The main area is titled 'Ambiente: LOCAL' and contains the instruction 'Crie variáveis que mudam conforme onde a API está rodando.' Below this is a table with three columns: 'Variable', 'Initial value', and 'Current value'.

Variable	Initial value	Current value
baseUrl	https://postman-echo.com	https://postman-echo.com
usuarioTeste	pedro.local	pedro.local
apiKey	NUNCA colocar em Initial	valor local privado

Below the table, a green box contains the following text: 'Initial value pode sincronizar/compartilhar. Current value é o valor usado localmente. Para tokens e senhas, use Current value e não compartilhe dados sensíveis.'

Exemplo visual de ambiente LOCAL com baseUrl e outras variáveis.

Passo a passo no Postman

- No menu lateral, acesse Environments.
- Clique em + ou Create Environment.
- Crie um ambiente chamado LOCAL.
- Adicione a variável baseUrl.
- Em Current value, coloque https://postman-echo.com.
- Selecione o ambiente no canto superior direito antes de enviar a request.

Initial value x Current value

Initial value é o valor inicial/sincronizável. Current value é o valor usado localmente na execução. Para dados sensíveis, como token, senha e API key, prefira usar Current value e evite compartilhar esses valores.

4. Base URL: o primeiro passo para organizar requests

A Base URL é uma variável que guarda o começo do endereço da API. Ela evita que você precise alterar manualmente todos os endpoints quando a API muda de ambiente.

Usando Base URL em uma request



Separação entre baseUrl, endpoint e query params em uma request GET.

Tutorial prático com Postman Echo

Crie uma request com o método GET e use a URL abaixo:

```
{{baseUrl}}/get?cliente=Pedro&origem=postman
```

Com o ambiente LOCAL ativo e baseUrl = https://postman-echo.com, o Postman enviará:

```
https://postman-echo.com/get?cliente=Pedro&origem=postman
```

Na resposta, a Echo API deve devolver os argumentos recebidos. Assim você consegue conferir se os query params foram enviados corretamente.

Quando usar Base URL?

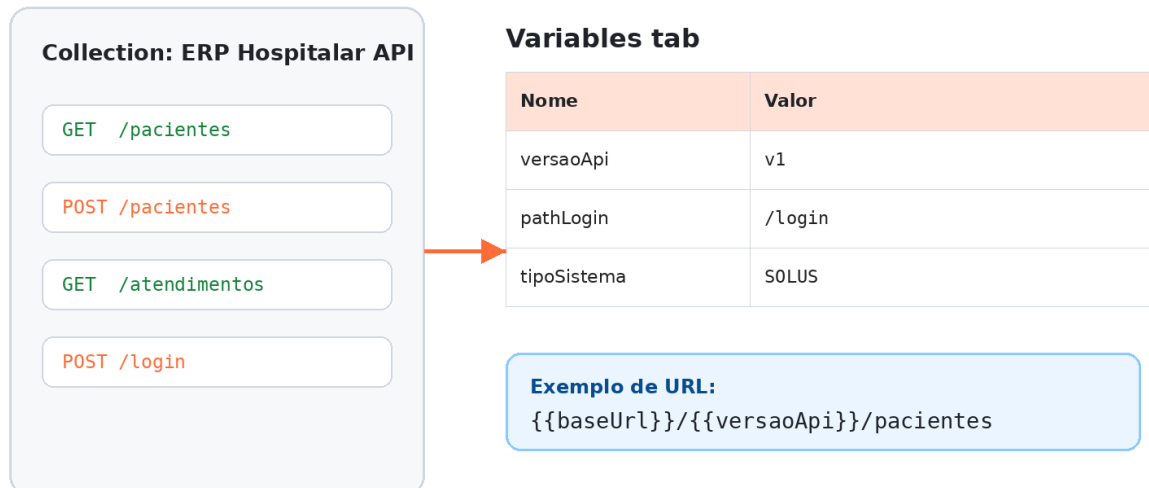
- Sempre que várias requests começam com o mesmo domínio.
- Sempre que você alterna entre LOCAL, HOMOLOG e PROD.
- Sempre que quer deixar a collection mais limpa e profissional.

5. Variáveis de Collection

Collection variables são boas para valores que pertencem à API como um todo e não necessariamente mudam por ambiente. Elas ficam dentro da collection e podem ser reutilizadas por todos os endpoints daquela collection.

Variáveis de Collection

Use para valores compartilhados por todos os endpoints da mesma collection.



Collection variables ajudam a compartilhar valores comuns entre endpoints.

Exemplos úteis

```
versaoApi = v1
pathLogin = /login
tipoSistema = SOLUS
```

Exemplo de URL usando ambiente + collection variable:

```
{{baseUrl}}/{{versaoApi}}/pacientes
```

Boas práticas

- Use nomes claros: baseUrl, versaoApi, usuarioTeste.
- Evite nomes genéricos como x, url ou teste.
- Não crie a mesma variável em muitos lugares sem necessidade, para evitar confusão de escopo.
- Documente a variável quando ela não for óbvia.

6. Dynamic variables e Faker: dados falsos automáticos

Dynamic variables são variáveis prontas do Postman que geram valores aleatórios. Elas usam a biblioteca Faker para criar nomes, e-mails, cidades, números, UUIDs e outros dados de exemplo.

Dynamic variables / Faker

Variáveis dinâmicas geram dados diferentes a cada envio da request.

Body - raw JSON

```
{
  "nome": "{{${randomFullName}}}",
  "email": "{{${randomEmail}}}",
  "cidade": "{{${randomCity}}}",
  "idExterno": "{{${guid}}}"
}
```

Envio 1

nome: Mariana Souza
email: mariana@example.com

Envio 2

nome: Lucas Pereira
email: lucas@example.org

Atenção: dados Faker são falsos. Não use como dados

Exemplo de body JSON usando variáveis dinâmicas. Cada envio pode gerar valores diferentes.

Exemplo de body para POST

```
{
  "nome": "{{${randomFullName}}}",
  "email": "{{${randomEmail}}}",
  "cidade": "{{${randomCity}}}",
  "idExterno": "{{${guid}}}"
}
```

Crie uma request POST para:

```
{{baseUrl}}/post
```

Na aba Body, selecione raw e JSON. Cole o JSON acima e clique em Send. A Postman Echo API devolverá o conteúdo enviado, permitindo validar os dados gerados.

Dynamic variables úteis

Variável	Gera	Exemplo de uso
{{\${randomFullName}}}	Nome completo falso	"nome": "{{\${randomFullName}}}"
{{\${randomEmail}}}	E-mail falso	"email": "{{\${randomEmail}}}"
{{\${randomCity}}}	Cidade falsa	"cidade": "{{\${randomCity}}}"
{{\${guid}}}	Identificador UUID/GUID	"id": "{{\${guid}}}"
{{\${timestamp}}}	Timestamp atual	"createdAt": "{{\${timestamp}}}"

7. Usando variáveis em URL, Params, Headers e Body

O Postman permite usar variáveis em várias partes da request. Isso deixa seu teste reutilizável e reduz manutenção.

Local	Exemplo	Quando usar
URL	{{baseUrl}}/get	Trocar ambiente sem reescrever endpoints.
Query Params	cliente={{nomeCliente}}	Filtrar ou enviar parâmetros simples.
Headers	Authorization: Bearer {{token}}	Autenticação e metadados da request.
Body JSON	"email": "{{\${randomEmail}}}"	Enviar payloads dinâmicos em POST/PUT/PATCH.

Exemplo completo de POST

```
POST {{baseUrl}}/post
Header:
Content-Type: application/json
```

```
Body:
{
  "nome": "{{${randomFullName}}}",
  "email": "{{${randomEmail}}}",
  "sistema": "{{tipoSistema}}"
}
```

Nesse exemplo, baseUrl vem do ambiente, tipoSistema pode vir da collection, e os dados de nome/e-mail são gerados dinamicamente pelo Faker.

8. Scripts: salvar valores automaticamente

Além de escrever variáveis manualmente, você pode criar ou atualizar variáveis usando scripts no Postman. Isso é útil quando uma request de login retorna um token e você quer usar esse token nas próximas requests.

Exemplo: salvar token de uma resposta

```
// Aba Tests da request de login
const json = pm.response.json();
pm.environment.set("token", json.token);
```

Depois disso, em outra request, você pode usar:

```
Authorization: Bearer {{token}}
```

Principais comandos

```
// Environment variable
pm.environment.set("baseUrl", "https://postman-echo.com");
pm.environment.get("baseUrl");

// Collection variable
pm.collectionVariables.set("versaoApi", "v1");
pm.collectionVariables.get("versaoApi");

// Variável local temporária
pm.variables.set("idTemporario", "123");
```

Regra mental: se o valor muda por ambiente, use environment. Se pertence à collection inteira, use collection variable. Se é só temporário dentro de um fluxo/script, use variável local.

9. Tutorial guiado: mini collection de treino

Agora monte uma mini collection para treinar os conteúdos das aulas.

Passo 1 - Crie a collection

- Nome: Estudo Postman - Variáveis.
- Descrição: Collection para praticar environments, baseUrl, collection variables e Faker.

Passo 2 - Crie o ambiente LOCAL

```
baseUrl = https://postman-echo.com
```

Passo 3 - Crie uma collection variable

```
origem = estudo-postman
```

Passo 4 - Request GET

```
{{baseUrl}}/get?origem={{origem}}&email={{$randomEmail}}
```

Passo 5 - Request POST

```
{{baseUrl}}/post
```

Body JSON:

```
{
  "origem": "{{origem}}",
  "nome": "{{$randomFullName}}",
  "email": "{{$randomEmail}}",
  "cidade": "{{$randomCity}}"
}
```

Passo 6 - Valide a resposta

Confira se a resposta da Echo API mostra os dados enviados. Se aparecer `{{baseUrl}}` literal na resposta ou a request falhar por URL inválida, provavelmente o ambiente não está selecionado ou a variável não existe.

10. Erros comuns e como resolver

Problema	Causa provável	Como resolver
Aparece <code>{{baseUrl}}</code> na URL sem substituir	Ambiente não selecionado ou variável com nome diferente.	Selecione o ambiente e confira maiúsculas/minúsculas.
Request vai para o ambiente errado	Ambiente ativo está errado.	Verifique o seletor de ambiente antes de enviar.
Token não funciona	Token expirado, salvo no escopo errado ou com espaço.	Reexecute login e confira Authorization.
Variável da collection não aparece	Criada em outra collection ou com nome diferente.	Abra a collection correta e revise a aba Variables.
Dados Faker não mudam	Sintaxe incorreta.	Use <code>{{randomEmail}}</code> , com \$ e duas chaves.

Checklist antes de pedir ajuda

- O ambiente correto está selecionado?
- A variável está escrita exatamente igual?
- Você colocou duas chaves antes e depois: `{{variavel}}`?
- O valor está no Current value?
- A URL final aparece correta ao passar o mouse sobre a variável?
- O método HTTP está correto: GET, POST, PUT ou DELETE?

11. Boas práticas para projetos reais

- Padronize nomes: use baseUrl, token, versaoApi, usuarioTeste.
- Crie ambientes separados para LOCAL, HOMOLOG e PROD.
- Evite variável global em projeto real; prefira escopos mais específicos.
- Nunca compartilhe token, senha, API key ou dados sensíveis de pacientes/clientes.
- Use Faker para massa de teste fictícia, não para simular dados reais de pessoas.
- Documente a collection com descrições em endpoints importantes.
- Use exemplos de resposta para ajudar outros analistas/devs a entenderem a API.
- Quando possível, salve tokens automaticamente via scripts na aba Tests.

Exemplo de organização profissional

Collection: ERP Hospitalar API

Variables:

versaoApi = v1

sistema = SOLUS

Environment: LOCAL

baseUrl = http://localhost:8080

token = <current value local>

Environment: HOMOLOG

baseUrl = https://homolog.exemplo.com

token = <current value local>

Environment: PROD

baseUrl = https://api.exemplo.com

token = <current value local>

Atenção em ambiente hospitalar: nunca use CPF, nome, carteirinha, atendimento ou dados reais de paciente em collection pública, print, documentação aberta ou ambiente compartilhado. Use nomes fictícios e dados Faker.

12. Resumo rápido para revisar antes da aula

Assunto	Resumo
Environment	Conjunto de variáveis para um contexto: LOCAL, HOMOLOG ou PROD.
Base URL	Variável que guarda o começo da URL da API. Ex.: <code>{{baseUrl}}</code> .
Collection variable	Variável compartilhada entre todas as requests da collection.
Dynamic variables	Variáveis prontas que geram dados aleatórios. Ex.: <code>{{randomEmail}}</code> .
Escopo	Define onde a variável existe e qual valor vence em caso de conflito.
Segurança	Não compartilhe tokens/senhas. Use Current value para dados sensíveis.

Exercício final

Crie duas environments: LOCAL e HOMOLOG. Em cada uma, configure uma baseUrl diferente. Depois crie uma mesma request usando `{{baseUrl}}`/get e alterne o ambiente ativo. O objetivo é perceber que a request não muda, apenas o valor da variável.

Fontes consultadas

Postman Docs - Reuse data with variables and environments in Postman:

<https://learning.postman.com/docs/use/send-requests/variables/variables-intro>

Postman Docs - Store and reuse values using variables:

<https://learning.postman.com/docs/use/send-requests/variables/variables>

Postman Docs - Group sets of variables using environments:

<https://learning.postman.com/docs/use/send-requests/variables/managing-environments>

Postman Docs - Reference variables in scripts:

<https://learning.postman.com/docs/tests-and-scripts/write-scripts/postman-sandbox-reference/pm-variables>

Postman Docs - Dynamic variables / Faker:

<https://learning.postman.com/docs/tests-and-scripts/write-scripts/variables-list>

Postman Docs - Postman Echo API: <https://learning.postman.com/docs/reference/developer-resources/echo-api>

Postman Docs - Parameters and body data in requests:

<https://learning.postman.com/docs/use/send-requests/create-requests/parameters>

Observação: as imagens deste PDF foram recriadas como diagramas didáticos próprios, inspirados nos conceitos da documentação oficial, para evitar prints inconsistentes e melhorar a leitura.